Boolean Secret Sharing

Elisabeth Oswald

ROADMAP

Conceptual view
- Defining secret sharing
- Defining security
- Splitting the secret

Boolean secret sharing
- (2,2) Boolean secret sharing
- (d,d) Boolean secret sharing

History

Secret sharing was a key topic already around the “birth” of modern cryptography in 1979.

Shamir and Blakley both independently developed schemes that enable to share out a secret (the main application was to distribute a key/password/passphrase).

Blakley’s scheme can be understood as a variation of Shamir’s scheme; it is less efficient.

Our main focus will be on Boolean secret sharing and Shamir secret sharing.

SECRET SHARING: FORMALLY

We want a scheme in which we can “distribute” information about a secret to multiple parties.

Only if “sufficiently” many parties come together, they can reveal the secret.

For instance, five generals hold shares of a secret code to launch as super-duper weapon. Only if at least four generals agree, and come together, can they reveal the secret.

No single general, no pair of generals, and no triplet of generals learns anything about the secret though.

SECRET SHARING: FORMALLY

Secret Sharing Scheme

Given a message space $\mathcal{M}$, a threshold value $t \in \mathbb{N}$, and a number of shares $n \in \mathbb{N}$, with $t \leq n$.

A (t, n) **threshold secret sharing scheme** consists of two algorithms:

`Share`: takes a message $m \in \mathcal{M}$ as input, and outputs an n-tuple of randomly selected shares $\mathbf{s} = (s_1, \ldots, s_n)$.

`Reconstruct`: takes a subset of t or more shares as input, and produces a message m' as output.

The example from the previous slide would therefore map to a $(4, 5)$ threshold secret sharing scheme.

Notice the word "randomly" in `Share`: it implies that this is a *probabilistic algorithm*.

SECRET SHARING: FORMALLY

Correctness for secret sharing

A (t, n) threshold secret sharing scheme is **correct** if, for all authorised sets of user $\mathcal{U}$, s.t. $|\mathcal{U}| \geq t$), and for all $\mathbf{s} \leftarrow \texttt{Share}(m)$, we have that $\texttt{Reconstruct}(\{s_i, i \in \mathcal{U}\}) = m$.

In the definition we only require that `Reconstruct` returns something that is from the message space. Obviously, the minimum requirement for any concrete instance, if "enough authorised users" come together, the secret sharing scheme return the secret correctly.

SECRET SHARING: SET UP

Why use/have this weird/complex notation for a “set of authorised users”?

In practice we might want that some parties have more influence than others, so we give them more shares.

For instance, in addition to the five generals, we might also have a president who has some authority in launching the super-duper weapon. Suppose that without his blessing nobody should be allowed to fire the weapon (but they cannot fire the weapon without the support of at least 4 generals either).

How could we do this in a (t, n) threshold secret sharing scheme?

ROADMAP

Conceptual view
- Defining secret sharing
- Defining security
- Splitting the secret

Boolean secret sharing
- (2,2) Boolean secret sharing
- (d,d) Boolean secret sharing

SECURE SECRET SHARING

Perfectly secure secret sharing

Let Σ be a (t, n) threshold secret sharing scheme with message space $\mathcal{M}$. We say that Σ is **perfectly secure** if for all choices of messages $m_0, m_1 \in \mathcal{M}$, no unauthorised subset of users $\mathcal{U}$ can distinguish between the shares of m_0 and m_1.

Notice that this definition corresponds to the "indistinguishability" of ciphertext notion (symmetric encryption).

VISUALISING THE DEFINITION

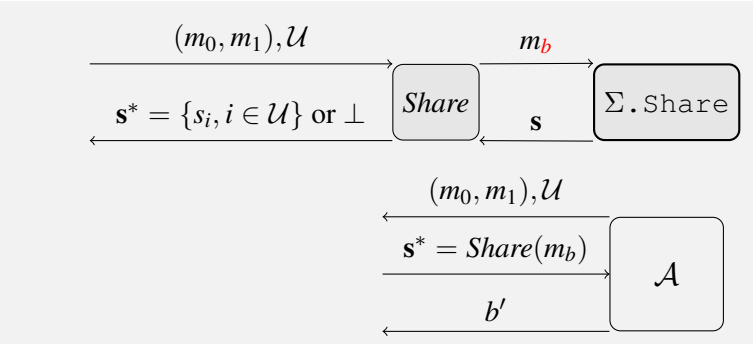


Figure: Exp ind-shares: A wins if $b' = b$.

$\mathcal{A}$ interacts with a function called *Share* that acts as interface to `Share` of considered secret sharing scheme. It checks if $\mathcal{U}$ is an unauthorised subset.

`Share` returns n shares; *Share* passes i for subset of users $\mathcal{U}$ to $\mathcal{A}$; otherwise returns $\perp$.

VISUALISING AN EXAMPLE

Let Σ be a $(2, 2)$ secret sharing scheme:

- there are two users u_1, u_2
- the set of authorised users is $\{u_1, u_2\}$
- the unauthorised subsets are $\{u_1\}$ and $\{u_2\}$.

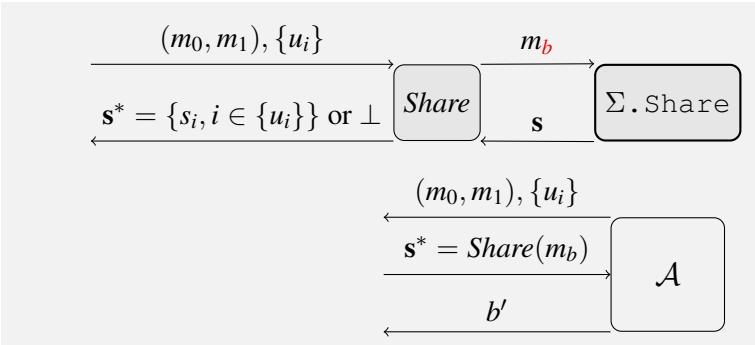


Figure: Exp ind-shares: A wins if $b' = b$.

The Share function returns $\perp$ if $\mathcal{A}$ supplies $\mathcal{U} = \{u_1, u_2\}$: this is because any authorised set of users can reconstruct, and therefore tell which message was shared out.

The LoR Version of the Definition for (2,2) scheme

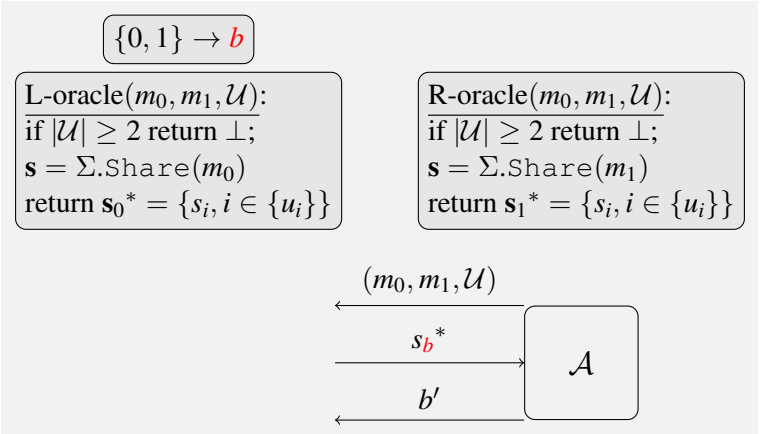


Figure: Win if $b' = b$.

Over to You

Can you come up with a Real or Random Game for this definition?

ROADMAP

Conceptual view
- Defining secret sharing
- Defining security
- Splitting the secret

Boolean secret sharing
- (2,2) Boolean secret sharing
- (d,d) Boolean secret sharing

SHARING IS NOT SPLITTING

Some resources use the term *secret splitting* instead of secret sharing.

Take this "literally", so consider an approach like this:

```
1  m = m_1||...||m_n
2  generate random permutation J;
3  for i = 1 to n do
4     s_i ← m_{J(i)}
5  return (s_1||...||s_n);
```

Listing: An insecure $\Sigma.\texttt{Share}(m)$

Notice that the definition from before requires that shares are "random": thus the random assignment from chunks to shares. Obviously this makes reversing the sharing "difficult".

Over to You

Secret splitting as defined in the previous slide is obviously insecure: why?

Can you show more formally, i.e. by using the security definition (and associated game) that this scheme is insecure?

ROADMAP

Conceptual view
- Defining secret sharing
- Defining security
- Splitting the secret

Boolean secret sharing
- (2,2) Boolean secret sharing
- (d,d) Boolean secret sharing

An $(2,2)$ Boolean secret sharing scheme

We assume that the purpose of the scheme is to work on bit strings. Therefore, we will use the bit-wise addition modulo 2 (aka Boolean addition, aka exclusive-or) to create two shares for an l-bit input message.

```
1  def B22.Share(m):
2      s_1 → {0,1}^l
3      s_2 = s_1 ⊕ m
4      return (s_1, s_2)
```

Listing: Boolean (2,2) secret sharing: Share function

```
1  def B22.Reconstruct(s_1, s_2):
2      return (s_1 ⊕ s_2)
```

Listing: Boolean (2,2) secret sharing: Reconstruct function

Set of authorised users, correctness

A $(2, 2)$ secret sharing scheme implies a situation where secret information is distributed among exactly 2 users/parties. Only with both users, Reconstruct is possible.

Sets of unauthorised users are $\{\{1\}, \{2\}\}$ (i.e. no single user should learn anything about s).

The only authorised set is thus $\{1, 2\}$ (it includes both users).

Correctness follows immediately from the construction: the reconstruct function computes $s_1 \oplus s_2 = s_1 \oplus (s_1 \oplus m) = m$.

EXAMPLE

Suppose that two executives wish to share the secret PIN to a company safe. The PIN has been chosen as 0123. They wish to use a Boolean secret sharing scheme.

We first need to represent the PIN as a binary string. There are multiple ways to do that: e.g. interpret 0123 as a decimal number, and then covert this to binary; or interpret 0123 as a string of digits, and then covert this to binary.

We choose the latter and represent 0123 is represented as 00011011.

`B22.Share` might sample a random 8 bit string as $s_1 = 00101110$. Therefore $s_2 = s_1 \oplus 00011011 = 00110101$.

The secret PIN can be reconstructed via $s_1 \oplus s_2 = 00101110 \oplus 00110101 = 00011011$.

Security of (2,2) Boolean secret sharing: intuition

s_1 is just a random string that is independently chosen of the message.

s_2 is an exclusive-or between the message and the random string s_1: essentially it is a one-time pad encryption of m.

We know the one-time pad is perfectly secret. Ergo s_2 on its own does not reveal any information about m.

In the next slides, we will prove security in multiple ways, that all boil down to this intuition.

SECURITY OF (2,2) BOOLEAN SECRET SHARING: MORE FORMAL

Case 1: $\mathcal{A}$ (m, m'), and $\mathcal{U} = \{1\}$.

Thus $\mathcal{A}$ sees s_1.

Suppose that m was selected: then we need to show that $\Pr[s_1|m] = \Pr[s_1]$. This holds because s_1 is just a random number that gets selected independently of m. The probability $\Pr[s_1|m] = 1/2^l$.

Suppose that m' was selected: then we need to show that $\Pr[s_1|m'] = \Pr[s_1]$. This holds because s_1 is just a random number that gets selected independently of m'. The probability $\Pr[s_1|m'] = 1/2^l$.

Therefore the probability distribution of the output s_1 is the uniform distribution (all elements are equally likely) independent of the choice of m, m'. An adversary can therefore not tell whether m or m' was shared out.

SECURITY OF (2,2) BOOLEAN SECRET SHARING: MORE FORMAL

Case 2: $\mathcal{A}$ (m, m'), and $\mathcal{U} = \{2\}$.

Thus $\mathcal{A}$ sees s_2.

Remember from one-time pad encryption that the distribution of $k \oplus m$, when k is uniformly distributed, is the uniform distribution.

Suppose m was shared: $s_2 = s_1 \oplus m$. This means that $\Pr[s_2|m] = \Pr[s_2 = s_1 \oplus m] = \Pr[s_1 = s_2 \oplus m] = 1/2^l$. The distribution of the second share $\Pr[s_2|m]$ is the uniform distribution.

Suppose m' was shared: $s_2 = s_1 \oplus m'$. This means that $\Pr[s_2|m'] = \Pr[s_2 = s_1 \oplus m'] = \Pr[s_1 = s_2 \oplus m'] = 1/2^l$; so is also uniform.

Thus the distribution of s_2 is always uniform, and the adversary cannot tell whether m or m' was shared out.

SECURITY ARGUMENT USING CALLS TO ORACLES

We can frame the same logic in the context of calls to Left/Right oracles and show that their output distributions are indistinguishable.

We can do this by showing that the left oracle can be "changed" to behave like the right oracle, without actually changing its output distribution.

Below is the code for the left and right oracles for the Boolean share scheme, where I have already inlined the call to the Σ.`Share` function (i.e. $B22$.`Share`). (Perhaps look back at the previous slide with the LoR version of the game.)

```
1  L-oracle(m_L, m_R):
2      if U ≥ 2 return ⊥
3      s_1 ←{0,1}^l
4      s_2 = s_1 ⊕ m_L
5      return {s_i | i ∈ U}
```

```
1  R-oracle(m_L, m_R):
2      if U ≥ 2 return ⊥
3      s_1 ←{0,1}^l
4      s_2 = s_1 ⊕ m_R
5      return {s_i | i ∈ U}
```

SECURITY ARGUMENT WITH ORACLES, CONT.

```
1   L-oracle(m_L, m_R):
2       if U ≥ 2 return ⊥
3       if U = {1}
4           s_1 ←{0,1}^l
5           s_2 = s_1 ⊕ m_L
6           return s_1
7       if U = {2}
8           s_1 ←{0,1}^l
9           s_2 = s_1 ⊕ m_L
10          return s_2
```

```
1   L-oracle(m_L, m_R):
2       if U ≥ 2 return ⊥
3       if U = {1}
4           s_1 ←{0,1}^l
5           s_2 = s_1 ⊕ m_R
6           return s_1
7       if U = {2}
8           s_1 ←{0,1}^l
9           s_2 = s_1 ⊕ m_L
10          return s_2
```

First, we expand the if statement, and create "straight line" code.

Because s_2 is not returned if $\mathcal{U} = \{1\}$, we can replace m_L with m_R.

SECURITY ARGUMENT WITH ORACLES, CONT.

If $\mathcal{U} = \{2\}$ then s_2 will be returned. The share s_2 is the one-time pad encryption of the message m_L.

We know that the ciphertext distribution of the one-time pad is the uniform distribution: all ciphertext are equally likely, and independent of the input message. Hence if we change the message from m_L to m_R we do not change the distribution of s_2: so we may swap m_L to m_R.

```
1   L-oracle(m_L, m_R):
2       if U ≥ 2 return ⊥
3       if U = {1}
4           s_1 ←{0,1}^l
5           s_2 = s_1 ⊕ m_R
6           return s_1
7       if U = {2}
8           s_1 ←{0,1}^l
9           s_2 = s_1 ⊕ m_R
10          return s_2
```

SECURITY ARGUMENT USING CALLS TO ORACLES, CONT.

Finally we merge the if function branches again to come back to our original pseudo-code:

```
1  L-oracle(m_L, m_R):
2      if U ≥ 2 return ⊥
3      s_1 ← {0,1}^l
4      s_2 = s_1 ⊕ m_R
5      return {s_i | i ∈ U}
```

```
1  R-oracle(m_L, m_R):
2      if U ≥ 2 return ⊥
3      s_1 ← {0,1}^l
4      s_2 = s_1 ⊕ m_R
5      return {s_i | i ∈ U}
```

Clearly the Left oracle now does exactly what the Right oracle does, and therefore no adversary can distinguish between them.

Generalising Boolean secret sharing

It is straightforward to replace “addition modulo 2” with “addition modulo m”.

It is also straightforward to extend to the (n, n) case.

It is doable to extend to the $(2, n)$ case by constructing $(2, 2)$ secret sharing schemes for all pairs of users.

Going to a (t, n) scheme is not nice; this is why we will look at Shamir Secret Sharing later.

ROADMAP

Conceptual view
- Defining secret sharing
- Defining security
- Splitting the secret

Boolean secret sharing
- (2,2) Boolean secret sharing
- (d,d) Boolean secret sharing

A (d, d) Boolean secret sharing scheme

We assume that the scheme is to work on bit strings (aka finite field elements of F_{2^l}).

```
1  def Bdd.Share(v,d):
2      for i = 1 upto d − 1
3          v_i ←{0,1}^l
4      v_d = v_1 ⊕ v_2 ... v_{d−1} ⊕ v
5      return {v_1, v_2, ..., v_d}
```

Listing: Boolean (d, d) secret sharing: Share function

```
1  def Bdd.Reconstruct({v_1, v_2, ..., v_d}):
2      return (v_1 ⊕ v_2 ⊕ ··· ⊕ v_d)
```

Listing: Boolean (d, d) secret sharing: Reconstruct function

Over to You

Suppose that three executives wish to share the secret PIN to a company safe. The PIN has been secure chosen as 0123. They wish to use a Boolean secret sharing scheme. Given an instance of a correct sharing, and reconstruction.

Security argument: Left vs Right World

```
L-oracle(m_L, m_R, d, U):
 if |U| > d − 1:
     return ⊥
 for i = 1 upto d − 1
     v_i ←{0,1}^l
 v_d = v_1 ⊕ v_2 ... v_{d−1} ⊕ m_L
 return {v_i | i ∈ U}
```

Listing: Left World

```
R-Oracle(m_L, m_R, d, U):
 if |U| > d − 1:
     return ⊥
 for i = 1 upto d − 1
     v_i ←{0,1}^l
 v_d = v_1 ⊕ v_2 ... v_{d−1} ⊕ m_R
 return {v_i | i ∈ U}
```

Listing: Right World

We must convincingly argue that the Left world and the Right world are indistinguishable from one another: we "rewrite" the Left world, using changes that cannot change the outputs/distribution, so it is the Right world.

Security argument: Left vs Right World, cont.

```
1  L-oracle(m_L, m_R, d, U):
2   if |U| > d − 1:
3      return ⊥
4   if v_d ∉ U:
5     for i = 1 upto d − 1
6        v_i ←{0,1}^l
7     v_d = v_1 ⊕ v_2 ... v_{d−1} ⊕ m_L
8     return {v_1, ..., v_{d−1}}
9   if v_d ∈ U:
10    for i = 1 upto d − 1
11       v_i ←{0,1}^l
12    v_d = v_1 ⊕ v_2 ... v_{d−1} ⊕ m_L
13   return {v_i | i ∈ U}
```

- First case: the adversary asks for indices that do not include the final share v_d: therefore, m_L can be changed to m_R.
- Second case, the adversary's request includes the share v_d: because v_d is independent of m_L, we may change m_L to m_R.

We use exactly the same arguments as in the $(2,2)$ Boolean Secret Sharing scheme.

Over to You

Can you show how the proof argument in the RoR version of the game works?